

The `sumDivConq()` function first checks the `numbers` array for having either zero or one number in it. These trivial base cases are easy to sum since they require no addition: return either 0 ❶ or the lone number in the array ❷. Everything else is a recursive case; the middle index of the array is calculated ❸ so that separate recursive calls with the left half and right half of the `numbers` array are made. The sum of these two return values becomes the return value for the current `sumDivConq()` call ❹.

Because of the associative nature of addition, there's no reason an array of numbers must be added sequentially by a single computer. Our program carries out all operations on the same computer, but for large arrays or calculations more complicated than addition, our program could send the halves to other computers to complete. The problem can be divided into similar subproblems, which is a huge hint that a recursive approach can be taken.